

Technical Assessment: AI Engineering

A take-home exercise in building software that incorporates AI — Python and React

Purpose

This role involves building tools that incorporate AI: applications where a language model, equipped with tools, does real work for a user. This assessment asks you to build a small example of exactly that.

Two distinct skills are on show here, and we care about both. The first is **AI engineering** — designing the agent, its tools, its prompts and its failure handling. The second is **AI-assisted development** — how you use AI tooling to build the thing itself. You will submit the codebase along with the artefacts of your AI sessions, then walk us through your approach in a short presentation.

The brief

Build a web application in which a user submits a task in natural language, and a Python backend completes it by running an **agent loop**: a language model equipped with tools sourced from at least one **MCP server**. The React frontend shows the user the outcome — and, importantly, what the agent did to get there.

The use case is your choice. Some examples of the right shape and size:

- Answering questions about a public GitHub repository, using GitHub's MCP server.
- Answering questions that require live web content, using a fetch or search MCP server.
- Answering questions about a library or API, using a documentation MCP server such as Context7.
- Or build your own MCP server for the task — a small, well-designed server of your own is just as welcome as a public one.

These are suggestions, not requirements — choose something else if it interests you more. Two constraints apply regardless:

CONSTRAINTS

- The agent must consume its tools **over MCP** — from a public server, one you built yourself, or a mix. Tools wired directly into the backend without the protocol do not satisfy the requirement.
- Tool selection must be the **model's decision**, made within a bounded agent loop. A hard-coded sequence of tool calls with the model summarising at the end does not satisfy the requirement.

Backend

Use Python; the web framework is your choice. The backend should:

- Expose an API consumed by the frontend.
- Run the agent loop: connect to the MCP server, present its tools to the model, execute the calls the model makes, and feed results back — supporting multiple tool calls per task.
- Bound the loop sensibly: an iteration cap and timeouts, at minimum.
- Fail well. An unreachable MCP server, a tool call that errors mid-task, or a model failure should each produce a meaningful state the frontend can present — not a hang or a stack trace.
- Take secrets from the environment. Nothing sensitive committed to the repository.

Frontend

Use React; TypeScript is encouraged. The frontend should let a user:

- Submit a task and see its progress and result.
- See the agent's tool activity: for each call, the tool name, the input, and a summary of the result. A user should be able to look at a completed task and understand what the agent did and why the answer is what it is.
- See sensible states when things fail.

Model access

Use your own API key with any mainstream model provider; a small, inexpensive model or locally-hosted model is entirely acceptable. If you do not have API access, you may substitute a **clearly signposted mock model adapter** behind the same interface as the real one — but the MCP integration must still be real, so in that case choose an MCP service that does not itself require paid access, or build your own.

Testing

Write automated tests where they earn their place. The agent loop exercised against a fake model and fake tool set is a natural target: loop bounds, error paths and result handling can all be verified without network access or spend.

Working with AI tools

Use AI tooling throughout the build itself — coding agents, editor assistants, chat models, whatever your normal workflow looks like. Two things matter to us:

- **Keep your working as you go.** Save your agent instruction files, your significant prompts, and transcripts or summaries of your sessions while you work. Reconstructing them afterwards is difficult, and the genuine record is far more interesting to us than a tidied one.
- **You are the author.** We will review the codebase as your work, to the standard of code you would put into production. Generated code you did not read is still your code.

Deliverables

- 1 **The repository**, as a git repo with its history intact. Commits that tell the story of how the work developed are far more useful to us than a single final commit.
- 2 **Your AI session artefacts**, in an `ai/` directory in the repo. Include your agent instruction files (for example `CLAUDE.md`, `.cursor/rules` or equivalent), the significant prompts you used, transcripts or session summaries, and a brief note on which tools and models you worked with. This is a required part of the submission, not an optional extra.
- 3 **A README** covering how to run the application (including required environment variables and how to enable the mock adapter if you built one), your chosen use case and MCP service, key decisions, known limitations, and what you would do next with more time.

The presentation

You will present your work back to us in a 15-minute session followed by questions. A live demo, slides, or a mix of the two — whichever suits you. We would like you to cover:

- Your chosen use case and MCP service, and why.
- The design of the agent: the system prompt, how tools are presented to the model, how the loop is bounded, and how failures are handled.
- How you directed your own AI tools during the build: instructions, prompts and iterations.
- What the AI got wrong — in the product or in the build — how you caught it, and what you did about it.

We are more interested in your judgement than in a feature tour. The most valuable ten minutes you can give us is the story of where you intervened.

Time expectations

This exercise is scoped for around **4 hours** for a developer familiar with agent development and fluent in AI-assisted workflows. Please do not substantially exceed that. If you run out of time, cut scope deliberately and record what you cut and why in the README — a well-judged cut is a positive signal, not a negative one.

What we assess

- **The design of the agent** — prompts, tool handling, loop bounds and failure behaviour.
- **How the system treats what it cannot trust** — a language model given external tools is handling content from outside your control, and we will look at how the design accounts for that.
- **The quality and consistency of the codebase as a whole** — we read it as production code carrying your name.
- **The transparency of the frontend** — whether it faithfully shows what the agent actually did.
- **How you directed and verified your AI tools during the build**, as evidenced by your artefacts, your commit history and your presentation.

We deliberately do not publish a marking checklist. Treat the submission as you would treat production work, and it will speak for itself.